

Bridge Day 12 INSTRUCTOR KEY

For Loops, Range Bounds, Counters, and Accumulators

Learning sequence: Predict - Trace - Explain - Test - Interpret

Tuesday, July 21, 2026 • 20 points • target 19 minutes

Name		UIN		Section	
------	--	-----	--	---------	--

Assessment boundary and evidence source

Contract: 07a04826c975

Cumulative foundations: input conversion, for/range, exclusive stop, accumulator, conditional counter

Scaffold level: complete fixed-count data program

Evidence source: the matching v5 workbook readiness/evidence pages and VIP activities 18.18, 18.20.

Show intermediate state for trace credit. Program credit requires one coherent solution, a stated assumption when the prompt leaves one implicit, and at least one test whose expected result can distinguish correct from incorrect logic.

Item 1. Trace every range-driven state update. - 5 points

```
total = 0
count = 0

for number in range(2, 10, 2):
    total = total + number
    if total >= 10:
        count = count + 1
    print(number, total, count)
```

Question: List range values, then give number, total, and count after every pass and all exact output.

Model response: range produces 2, 4, 6, 8. States and output are 2 2 0; 4 6 0; 6 12 1; 8 20 2. The counter tests the updated running total on each pass.

Item 2. Repair an excluded-stop defect. - 3 points

```
limit = 4
total = 0

for number in range(1, limit):
    total = total + number

print(total)
```

Question: The intent is 1 through limit inclusive. Give actual, intended, cause, repair, and one smallest boundary test.

Model response: The actual total is $1 + 2 + 3 = 6$; the intended total is 10. range excludes its stop, so repair with `range(1, limit + 1)`. For limit 1, the original loop gives 0 while the repaired loop gives 1.

Complete program - 12 points

- Read sample_count as an integer and threshold as a float.
- If sample_count is not positive, print invalid and request no readings.
- Otherwise read exactly sample_count floating-point readings.
- Compute total, average, and the count of readings at or above threshold.
- Print Total to one decimal, Average to two decimals, and At or above as an integer on separate labeled lines.

Program workspace

Model program

```
sample_count = int(input())
threshold = float(input())

if sample_count <= 0:
    print("invalid")
else:
    total = 0.0
    at_or_above = 0
    for sample in range(sample_count):
        reading = float(input())
        total = total + reading
        if reading >= threshold:
            at_or_above = at_or_above + 1
    average = total / sample_count
    print(f"Total: {total:.1f}")
    print(f"Average: {average:.2f}")
    print(f"At or above: {at_or_above}")
```

Test input, expected result, and why this test matters

Reasoning and test evidence The positive-count guard prevents invalid repetition and division by zero. Both summary states are initialized once before the loop; average is computed after all readings are processed. For count 3, threshold 4, and readings 3.5, 4, 5, output is Total: 12.5, Average: 4.17, and At or above: 2. Equality at the threshold is counted.

Scoring guidance: 2 guard/input, 2 initialization, 3 exact loop/repeated input, 2 total/count logic, 1 average/output, 2 boundary test.